Kiến trúc giải pháp & vận hành production (Solution Architecture)

Sản phẩm / hệ thống	Crawler & ingest bài viết web (RSS + Sitemap) lên PostgreSQL
Phiên bản kiến trúc	**Scope 1** — production-ready tại quy mô nhỏ, ưu tiên kiểm soát chi phí & vận hành
Độc giả	Solution / Cloud Architect, DevOps, chủ hệ thống
Căn cứ triển khai	Mã nguồn `src/crawlerdemo/`, `infrastructure/terraform/`, `infrastructure/aws/lambda_ingester/`

Mục tiêu tài liệu: Một **báo cáo kiến trúc** (phạm vi nghiệp vụ + kỹ thuật cụ thể: crawl gì, cơ chế, dung lượng/giới hạn, dữ liệu ra vào) đồng thời **map 1:1** Terraform & CI/CD trong repo.

Khái niệm	Vị trí trong repo
Môi trường triển khai	`infrastructure/terraform/environments/demo/`
Modules tái sử dụng	`infrastructure/terraform/modules/{networking,security,queue,storage,lambda,worker,observab`
Mã ứng dụng / Lambda	`infrastructure/aws/lambda_ingester/`, worker Docker tại repo root
CI/CD	`.github/workflows/terraform-plan.yml`, `.github/workflows/deploy-aws.yml`

Prefix tài nguyên (mặc định): `project=crawler`, `environment=demo` → tên Terraform/local: `crawler-demo-*`.

Mục lục (11 phần — hệ thống chuẩn SA)

1. Phạm vi bài toán & yêu cầu phi chức năng (NFR) — *báo cáo scope: crawl gì, cơ chế, nguồn URL, giới hạn batch/dung lượng, RDS/S3*
2. Kiến trúc tổng quan & danh mục thành phần
3. Luồng dữ liệu end-to-end
4. Phân tích từng dịch vụ — lý do & map Terraform
5. Mạng & an ninh (Network, IAM, KMS, mã hóa)
6. Hạ tầng dạng mã — Terraform (IaC)
7. CI/CD — GitHub Actions
8. Hoạch định năng lực & "toán" cấu hình
9. Sẵn sàng cao, chịu lỗi & điểm đơn lỗi (SPOF)

10. Khả năng quan sát — log, metric, cảnh báo, dashboard
11. Checklist vận hành production

Phụ lục: A. Bản đồ nhanh khái niệm → resource Terraform · B. Sự cố thường gặp & chỗ tra

---

1. Phạm vi bài toán & yêu cầu phi chức năng (NFR)

Phần này là **phát biểu phạm vi nghiệp vụ + kỹ thuật** đủ chi tiết để độc lập với code: *hệ thống làm gì, thu thập từ đâu, theo cơ chế nào, giới hạn dung lượng / tần suất / batch như thế nào, dữ liệu lưu ra sao.*

1.1 Tóm tắt điều hành

Hệ thống **định kỳ** đọc các **nguồn RSS và Sitemap công khai**, trích xuất các **bài viết** (định danh bằng URL đã chuẩn hóa), đóng gói vào **một hàng đợi message**, và **ghi idempotent** vào **PostgreSQL** nhờ **Lambda consumer**. Payload quá lớn so với giới hạn message được **offload (gzip)** sang **S3 (Claim Check)**. **Worker không ghi DB trực tiếp**; toàn bộ persist do tầng ingest. Dashboard web trên worker phục vụ **theo dõi** (phạm vi nhỏ), không phải API search công khai quy mô lớn.

1.2 Phát biểu bài toán (Problem statement)

Cần một pipeline **ổn định — có thể mở rộng theo hướng tăng nguồn** — với các ràng buộc:

- **Tách producer/consumer** để crawl không chặn ingest và ngược lại.
- **Không nhân bản bản ghi** khi message SQS được giao **ít nhất một lần** (at-least-once).
- **Tách poison/error** khỏi luồng nóng (DLQ).
- **Kiểm soát chi phí và độ phức tạp** phù hợp lab / MVP (Single-AZ DB, NAT đơn, giới hạn concurrency Lambda).

1.3 Miền dữ liệu (Domain model)

Khái niệm	Định nghĩa trong code	Lưu trữ đích (RDS)
Bài viết (ArticleIn)	Bản ghi crawl: `source`, `canonical_url`, `title`, `summary`, `published_at`	Bảng `articles` (`schema.sql` / DDL Lambda): `canonical_url` unique, `summary` TEXT (summary qua JSON bị **cắt 500 ký tự** khi serialize để giữ batch trong ngưỡng message — xem `src/crawlerdemo/models.py`)
Nguồn (source)	Nhãn dạng `rss:<host>` / `sitemap:<host>`	Cột `source` (VARCHAR 120)
Khóa nghiệp vụ	`canonical_url` sau **canonicalize**	`UNIQUE INDEX uq_articles_canonical_url`

1.4 Phạm vi thu thập: "crawl gì" và "từ đâu"

Phạm vi kỹ thuật: chỉ **RSS/Atom** (qua thư viện **feedparser**) và **XML Sitemap** (qua **BeautifulSoup** parser XML). **Không** crawl HTML trang chủ hay deep-link từ trong bài trong scope mặc định này — chỉ URL xuất hiện trong feed hoặc sitemap.

Nguồn mặc định (có thể ghi đè bằng biến môi trường JSON `CRAWLER_RSS_URLS` / `CRAWLER_SITEMAP_URLS` — xem `src/crawlerdemo/config.py`):

Loại	URL mặc định (đặt trong code)

RSS	`https://www.state.gov/rss/channels/prsreleases.xml`, `.../remarks.xml`, `.../briefings.xml`; `https://congbao.chinhphu.vn/cac-van-ban-moi-ban-hanh.rss`
Sitemap	`https://www.theguardian.com/sitemaps/news.xml`; `https://en.baochinhphu.vn/sitemap.xml`

Kỳ vọng pháp lý / điều khoản: Nguồn chỉ mang tính **demo kỹ thuật**; môi trường production thực tế phải **tuân robots.txt, ToS, tần suất hợp lý** và whitelist IP (NAT) nếu site yêu cầu.

1.5 Cơ chế thu thập (behavior)

Thành phần	Hành vi	File tham chiếu
RSS	GET feed → parse từng entry: `link`, `title`, `summary`/`description`, `published`/`updated`	`src/crawlerdemo/sources/rss.py`
Sitemap	GET sitemap → nếu **`<sitemapindex>`** thì **đệ quy** vào từng `<loc>` con (cho đến khi đủ `limit`); nếu **`<urlset>`** thì đọc `<url><loc>`, lấy `lastmod` nếu có, title suy luận từ URL nếu không có title riêng	`src/crawlerdemo/sources/sitemap.py`
Chuẩn hóa URL	`canonicalize_url(...)` trước khi enqueue	`src/crawlerdemo/normalize.py`
Lập lịch	**APScheduler** `interval`: mặc định **1800 s (30 phút)** một vòng crawl toàn bộ nguồn; **`max_instances=1`**, `coalesce=True` — không cho hai vòng chồng lấp trong cùng process	`src/crawlerdemo/worker.py`
Vòng crawl một lần	Mỗi vòng có **`trace_id`** (UUID) chung log cho mọi nguồn	`run_once()` trong `worker.py`
HTTP	`httpx`, timeout mặc định **20 s**/request; User-Agent mặc định `crawlerdemo/1.0`	`src/crawlerdemo/config.py`, `http.py`

1.6 Giới hạn khối lượng & kích thước (throughput / "dung lượng" trong thiết kế)

Tham số	Giá trị mặc định (ứng dụng)	Giá trị triển khai Terraform (demo)	Ý nghĩa
Số bài tối đa / mỗi URL nguồn / mỗi vòng	`max_items_per_source = 100`	`module.worker`: **100** (`environments/demo/main.tf`)	Trần cứng số item parse mỗi RSS URL hoặc nhánh sitemap (sitemapindex đệ quy vẫn chịu chung "remaining" limit).
Kỳ crawl	`interval_seconds = 1800`	`var.crawler_interval_seconds` (mặc định **1800**)	Tần suất chạy `run_once` trên worker.
Một message	**Toàn bộ** danh sách `ArticleIn` của **một** nguồn (một RSS URL hoặc một	—	Không phải 1 message / 1 bài;

SQS chứa gì	nhánh sitemap đã gom) được serialize JSON **một lần** (`send_batch`)		degree of batching theo **nguồn**.
Giới hạn SQS	AWS **256 KiB**/message (Standard)	—	Thiết kế cố tình **dưới** ngưỡng đó.
Ngưỡng Claim Check	**200 KiB** (`200 * 1024` bytes) trong `Settings`	**204800** (= 200 KiB) trong Terraform worker	Nếu `len(JSON UTF-8)` **>** ngưỡng **và** đã cấu hình `s3_raw_bucket`: gzip + put S3 + body SQS chỉ còn pointer `claim_check_s3_*`.
Cắt summary	`summary` trong JSON payload tối đa **500** ký tự	—	Giảm nguy cơ vượt 256 KiB khi batch ~100 bài (`models.py`).
Lambda xử lý batch từ queue	—	**10** records / invocation (ESM `batch_size`)	Song song ingest có giới hạn ESM (mặc định **5** batch).
Lưu trữ raw trên S3	—	Lifecycle **xoá object raw sau 30 ngày** (`raw_expiration_days` mặc định module storage)	Raw chỉ phục vụ Claim Check trước khi ingest; không phải archive dài hạn.
RDS storage	—	gp3 **20 GB** ban đầu, autoscale tối đa **100 GB** (mặc định module storage)	Giới hạn phình disk cho bảng `articles` và index.

1.7 Đầu ra & cam kết dữ liệu

- **Persist:** Lambda upsert/idempotent `INSERT ... ON CONFLICT (canonical_url) DO NOTHING` → bảng `articles`.
- **Dashboard:** FastAPI trên worker (cổng `web_port`, mặc định 8080) phía sau ALB — đọc DB để hiển thị; không mô tả chi API surface trong tài liệu này.

1.8 Các kịch bản kỹ thuật (A–E) — map kiến trúc / Terraform

ID	Trường hợp	Hành vi	Map Terraform / code
A	Payload nhỏ	Body SQS chứa JSON trực tiếp	`module.queue` `aws_sqs_queue.main`; Producer: worker IAM `sqs:SendMessage`
B	Payload lớn	Claim Check: gzip → `module.storage` **S3 raw** → pointer trên SQS	`aws_s3_bucket.raw`; `claim_check_threshold_bytes` = **204800** trong `environments/demo/main.tf`
C	Trùng URL	`ON CONFLICT DO NOTHING` + unique index	`lambda_function.py`; `infrastructure/aws/lambda_ingester/schema.sql`

D	Lỗi tạm	`ReportBatchItemFailures`	`aws_lambda_event_source_mapping.sqs` (`modules/lambda/main.tf`)
E	Poison	Sau **3** lần fail → **DLQ**	`max_receive_count = 3` (`modules/queue/main.tf`)

1.9 Ngoài scope (explicit)

- API search/query **công khai** quy mô lớn; CDN; xác thực người dùng cho dashboard (chỉ HTTP public qua ALB trong demo).
- Đọc **nội dung HTML full-text** từng trang bài (ngoài metadata feed/sitemap) — không nằm luồng crawl mặc định.
- **RDS Proxy**, read replica, multi-region active-active (xem §9).

1.10 NFR — bảng tóm tắt (Solution Architect)

NFR	Mục tiêu thiết kế	Hiện thực trong repo
Bảo mật	Phân tầng VPC, SG tối thiểu, IAM role, KMS, TLS cho SQS	`module.networking`, `module.security`, `module.queue` policies
Tính sẵn sàng	Compute worker **Multi-AZ**; queue/Lambda managed	ASG `private_subnet_ids` ≥ 2 AZ; SQS/Lambda
Durability dữ liệu	RDS + backup có retention; S3 lifecycle	`aws_db_instance.main`; S3 lifecycle raw/exports
Khả năng quan sát	Alarm "DLQ có message", Lambda error, RDS CPU, ASG max	`module.observability`
Hiệu năng / chi phí	Giới hạn concurrency Lambda ↔ RDS; S3 VPC endpoint giảm NAT	ESM `maximum_concurrency`; `aws_vpc_endpoint.s3`; NAT đơn
Tuân thủ vận hành	IaC + remote state; PR có `terraform plan`	`backend.tf` S3; workflow `terraform-plan.yml`

---

2. Kiến trúc tổng quan & danh mục thành phần

2.1 Sơ đồ logic (luồng chính + web)

```
Internet ──► ALB :80 ──► FastAPI dashboard trên EC2 worker :8080 (var.web_port, mặc
định 8080)
                                  │
EC2 ASG Worker (private, Multi-AZ) ──► SQS Standard ──► Lambda Ingester ──► RDS
PostgreSQL
                                       VT=1080s          batch=10
Single-AZ (demo)
                                       DLQ×3             max ESM=5
     │
▲
     └── Claim Check (lớn hơn ngưỡng) ──► S3 raw
──────────────────────────────────────┘
```

```
┌──────────────────┐   Crawl + Normalize   ┌──────────────┐   Event Source
┌──────────────────┐
│  EC2 ASG Worker  │──────────────────────►│ SQS Standard │──────────────────►│
Lambda Ingester│
│  min=1 max=2     │                       │ VT=1080, DLQ │                   │
batch=10, maxESM=5│
└──────────────────┘                       └──────────────┘
└──────────────────┘
       │ Claim Check                                                          │
UPSERT
       ▼                                                                      ▼
┌──────────────┐
┌──────────────┐
│ S3 raw bucket│                                                   │ RDS
Postgres │
└──────────────┘
└──────────────┘
       │
       └──► S3 exports bucket (csv/json downstream)
```

2.2 Nguyên tắc kiến trúc (SA)

- **Loose coupling:** SQS đệm giữa crawl và ingest.
- **At-least-once delivery + idempotency tại DB:** trùng retry không làm hỏng dữ liệu cuối.
- **Governor cho RDS:** giới hạn **đồng thời batch** Lambda qua **SQS event source mapping** (`maximum_concurrency`), không "mở full" account concurrency lên DB nhỏ.
- **Claim Check:** tránh vượt giới hạn kích thước message và giảm rủi ro payload lớn.

2.3 Danh mục thành phần → module Terraform

Thành phần	Module	File chính
VPC, subnet, IGW, NAT, route, **S3 Gateway endpoint**	`module.networking`	`modules/networking/main.tf`
KMS, Secrets, SG, IAM Lambda/EC2	`module.security`	`modules/security/main.tf`
SQS main + DLQ, TLS policy	`module.queue`	`modules/queue/main.tf`
RDS PostgreSQL, S3 raw/exports	`module.storage`	`modules/storage/main.tf`
Lambda + layer + ESM + log group	`module.lambda`	`modules/lambda/main.tf`
ECR, Launch Template, ASG, CPU scaling	`module.worker`	`modules/worker/main.tf`
SNS, alarms, dashboard	`module.observability`	`modules/observability/main.tf`
ALB + TG + listener + attach ASG	(trong env)	`environments/demo/main.tf` — **không** là submodule

2.4 Thứ tự phụ thuộc Terraform (`environments/demo/main.tf`)

1. `networking` → 2. `security` → 3. `queue` → 4. `storage` → 5. `lambda` → 6. `worker` → (ALB resources) → 7. `observability`

---

3. Luồng dữ liệu end-to-end

3.1 Bước 1 — Worker crawl (ứng dụng)

1. APScheduler kích hoạt chu kỳ crawl (`crawler_interval_seconds` — biến TF, mặc định **1800**).
2. Gọi RSS/Sitemap qua HTTP client (`httpx`).
3. Parse, canonicalize URL, cắt summary theo giới hạn.
4. Tạo danh sách bài (`ArticleIn`).

Map triển khai: User-data → Docker → image từ **ECR** (`modules/worker`); biến môi trường crawl/queue/S3/db wire từ `module.worker` + `demo/main.tf` .

3.2 Bước 2 — Đẩy hàng đợi

1. Serialize JSON.
2. Nếu vượt `claim_check_threshold_bytes` (**204800**): gzip → upload **S3 raw** → message chứa pointer.
3. Gửi **SQS Standard** (batch API).

Map: `aws_sqs_queue.main` ; worker IAM `worker_custom` (SQS Send, S3 Put raw, KMS).

3.3 Bước 3 — Lambda consume

1. ESM gọi Lambda với tối đa **10** record/batch (`sqs_batch_size`).
2. Pointer → đọc S3 + gunzip → parse.
3. **Một kết nối DB cho mỗi invocation** (thiết kế giảm flood connection).

Map: `aws_lambda_event_source_mapping.sqs` ; `aws_lambda_function.ingester` ; env `RDS_HOST` , `DB_*` (`modules/lambda/main.tf`).

3.4 Bước 4 — Persist idempotent

1. `INSERT ... ON CONFLICT (canonical_url) DO NOTHING` .
2. Record OK → xóa khỏi SQS (partial success semantics).
3. Record lỗi → nằm trong `batchItemFailures` .

3.5 Bước 5 — Định tuyến lỗi

- Sau **3** lần nhận thất bại → **DLQ** (`max_receive_count`).
- Alarm khi DLQ có message → **SNS email** (`module.observability`).

---

4. Phân tích từng dịch vụ — lý do & map Terraform

4.1 EC2 Auto Scaling Worker + ECR

SA — lý do	Chi tiết	Terraform

Crawl là workload **dài**, có lịch; cần container ổn định	t3.micro, ASG **1/1/2**, Multi-AZ private	`aws_ecr_repository.worker`, `aws_launch_template.worker`, `aws_autoscaling_group.worker`
Scale theo CPU nghiệp vụ	Out **70%** / In **40%**, **3×60s**	`aws_cloudwatch_metric_alarm.cpu_high/low`, `aws_autoscaling_policy.scale_out/in`
Pull image an toàn	IMDSv2 required	`metadata_options` trong `aws_launch_template.worker`

Tham số ASG (wired trong `environments/demo/main.tf`): `desired_capacity=1`, `min_size=1`, `max_size=2`.

4.2 SQS Standard + DLQ

SA — lý do	Chi tiết	Terraform
Decouple & durable buffer	Standard (throughput)	`aws_sqs_queue.main`
Poison isolation	DLQ, `maxReceiveCount=3`	`aws_sqs_queue.dlq`, `redrive_policy`
Tránh double consume khi Lambda chậm	**VT = 1080s** chọn để khớp timeout Lambda	`visibility_timeout_seconds` trên cả main và DLQ queue resource
Bảo vệ in-transit	TLS-only deny	`aws_sqs_queue_policy.main/dlq`
At-rest	KMS	`kms_master_key_id`

4.3 Lambda Ingester

SA — lý do	Chi tiết	Terraform
Event-driven ingest, không quản OS	Python 3.12, timeout **180s**, memory **256**	`aws_lambda_function.ingester`
VPC để tới RDS private	subnet private + `sg_lambda`	`vpc_config`
Partial retry	`ReportBatchItemFailures`	`aws_lambda_event_source_mapping.sqs`
Giới hạn song song ở ESM	Default **5** batches	`scaling_config.maximum_concurrency` ← `var.lambda_event_source_max_concurrency`
Reserved concurrency	Account/quota — để **`null`** mặc định	`reserved_concurrent_executions`

Lưu ý tính nhất quán doc/code: comment đầu `environments/demo/main.tf` có thể khác phiên bản cũ ("Reserved=50"). **Nguồn sự thật:** `lambda_reserved_concurrency` trong `variables.tf` (**mặc định `null`**) và `lambda_event_source_max_concurrency` (**mặc định `5`**).

4.4 RDS PostgreSQL

SA — lý do	Chi tiết	Terraform
Source of truth quan hệ + constraint	`canonical_url` unique	Ứng dụng/schema; RDS `aws_db_instance.main`
Scope 1 chi phí	**Single-AZ**	`module.storage` + **override** `db_multi_az = false` trong `environments/demo/main.tf`
Vận hành	PI, enhanced monitoring, logs export	`aws_db_instance.main`

4.5 S3 (raw + exports)

Bucket	Mục đích	Terraform
Raw	Claim Check gzip, lifecycle expire	`aws_s3_bucket.raw`, `aws_s3_bucket_lifecycle_configuration.raw`
Exports	csv/json, versioning + transition IA	`aws_s3_bucket.exports`, versioning + lifecycle

4.6 ALB + Web dashboard

- Public **HTTP 80** → target group → instance worker cổng **web** (`web_port` , mặc định **8080**).
- Health check `/health` — **200**.

Terraform: `aws_lb.web` , `aws_lb_target_group.web` , `aws_lb_listener.web_http` , `aws_autoscaling_attachment.web_tg` , SG `aws_security_group.web_alb` + rule ingress worker từ ALB (`environments/demo/main.tf`).

---

5. Mạng & an ninh (Network, IAM, KMS, mã hóa)

5.1 Kiến trúc mạng

Lớp	Vai trò	Terraform
Public	IGW, NAT (1 NAT — cost Scope 1), ALB	`aws_subnet.public`, `aws_nat_gateway.main`, ALB `aws_lb.web`
Private	Worker ASG, Lambda ENI	`aws_subnet.private`
DB	RDS — không public, không default route internet	`aws_subnet.db`, `aws_route_table.db` (không 0.0.0.0/0)
S3 Gateway	Tránh egress NAT cho S3	`aws_vpc_endpoint.s3` (gắn route table **private** và **db**)

5.2 Security Groups (theo **code thực tế**)

SG	Ingress / Egress	Ghi chú SA
`aws_security_group.worker`	Egress all	Crawl qua NAT; nhận traffic web từ ALB qua **rule riêng** ở env

`aws_security_group.lambda`	Egress all	ENI Lambda → RDS/S3/SQS (API)
`aws_security_group.rds`	**5432** từ **Lambda** và **Worker**	Worker chạy **FastAPI** truy cập DB; khác mô tả tối giản chỉ "Lambda only"
`aws_security_group.web_alb`	**80** từ Internet (0.0.0.0/0)	ALB public (Scope 1); production nên HTTPS + WAF (nâng cấp)

5.3 IAM

- **Lambda:** `AWSLambdaBasicExecutionRole` + VPC access + policy `lambda_custom` (SQS consume pattern `crawler-demo-*`, S3 Get raw, KMS, Secrets).
- **Worker:** SSM, ECR read, CloudWatch Agent + `worker_custom` (SQS send, S3 put raw, KMS).

5.4 KMS & bảo vệ dữ liệu

- **KMS CMK** `aws_kms_key.main` — SQS, S3, RDS, Secrets (rotation bật trên key).
- **SQS:** policy bắt TLS; **RDS/S3** encrypt at rest với KMS.
- **Secrets Manager:** `aws_secretsmanager_secret.db` (Lambda IAM có quyền đọc; **mật khẩu Lambda env cũng truyền qua TF** — cần cân nhắc hardening: chỉ Secrets tại runtime).

5.5 Truy cập vận hành

- **SSM Session Manager** trên worker role — không cần bastion SSH cho tác vụ điều tra.

---

6. Hạ tầng dạng mã — Terraform (IaC)

6.1 Cấu trúc thư mục

```
infrastructure/terraform/
├── modules/{networking,security,queue,storage,lambda,worker,observability}
└── environments/demo/
    ├── main.tf          # wiring module + ALB
    ├── variables.tf
    ├── outputs.tf
    ├── backend.tf       # remote state S3
    └── terraform.tfvars.example
```

6.2 Backend & state (production discipline)

Mục	Giá trị	File
Backend	S3, encrypt, **use_lockfile** (không cần DynamoDB lock table theo cấu hình hiện tại)	`environments/demo/backend.tf`
Bucket / key / region	`crawler-terraform-state-478111025341`, `demo/terraform.tfstate`, `ap-southeast-1`	`backend.tf` — **đổi nếu account khác**

6.3 Biến môi trường quan trọng

Biến	Mặc định / ý nghĩa	File
`aws_region`	`ap-southeast-1`	`variables.tf`

`aws_account_id`	Bắt buộc	`variables.tf`
`db_password`	`TF_VAR_db_password` (≥8 ký tự)	`variables.tf`
`db_instance_class`	`db.t3.micro`	`variables.tf`
`ec2_instance_type`	`t3.micro`	`variables.tf`
`lambda_reserved_concurrency`	`null`	`variables.tf`
`lambda_event_source_max_concurrency`	5	`variables.tf`
`alert_email`	SNS	`variables.tf`

Tham số "baked" trong `main.tf` (không phải biến): `visibility_timeout_seconds=1080`, `max_receive_count=3`, Lambda `timeout=180`, `memory=256`, `batch_size=10`, ASG `1/1/2`, `claim_check_threshold_bytes=204800`, `db_multi_az=false` (override module storage).

6.4 Output vận hành (`outputs.tf`)

`rds_endpoint`, `s3_raw_bucket`, `s3_exports_bucket`, `sqs_queue_url`, `sqs_dlq_url`, `lambda_function_name`, `ecr_repository_url`, `worker_asg_name`, `nat_gateway_ip`, `cloudwatch_dashboard_url`, `sns_alert_topic_arn`, `kms_key_arn`, `db_secret_arn`, `web_dashboard_url`.

6.5 Lệnh tiêu chuẩn

```
terraform -chdir=infrastructure/terraform/environments/demo init
terraform -chdir=infrastructure/terraform/environments/demo plan
terraform -chdir=infrastructure/terraform/environments/demo apply
terraform -chdir=infrastructure/terraform/environments/demo output
```

---

7. CI/CD — GitHub Actions

7.1 Mục tiêu pipeline (SA)

- **PR:** kiểm tra định dạng + validate + **plan** với AWS (OIDC), không merge mù.
- **Main:** test ứng dụng, validate Terraform, **build/push ECR**, **cập nhật Lambda**, **rolling refresh ASG**.

7.2 Workflow `terraform-plan.yml` (Pull Request)

| Trigger | `pull_request` → `main`, path `infrastructure/terraform/**` || Bước | `init` → `fmt -check` → `validate` → `plan` || Credentials | OIDC → `arn:aws:iam::<AWS_ACCOUNT_ID>:role/GitHubActionsRole` || Secret | `TF_VAR_db_password` từ `secrets.TF_VAR_DB_PASSWORD` |

7.3 Workflow `deploy-aws.yml` (Push `main` / `workflow_dispatch`)

Job	Phụ thuộc	Hành động	Map tài nguyên AWS
`test`	—	`pytest`	Ứng dụng Python

`terraform-validate`	—	`fmt` + `init -backend=false` + `validate`	`environments/demo`
`build-and-push`	test + terraform-validate	`docker build` → ECR tag `:sha` + `:latest`	`ECR_REPOSITORY: crawler-demo-worker` phải khớp `aws_ecr_repository.worker`
`deploy-lambda`	terraform-validate	zip `lambda_function.py` → `aws lambda update-function-code --publish`	`LAMBDA_FUNCTION_NAME: crawler-demo-ingester`
`deploy-worker`	build-and-push	`start-instance-refresh` Rolling	`ASG_NAME: crawler-demo-worker-asg`

Điều kiện production: tên trong workflow phải **trùng** output Terraform (`crawler-demo-*` với default `project` / `environment`). Đổi project/env → cập nhật **env** trong workflow (hoặc parameterize bằng secret).

7.4 Thiết lập một lần (tóm tắt)

- IAM OIDC trust `token.actions.githubusercontent.com`, role `GitHubActionsRole` với quyền ECR, Lambda update, ASG refresh, (plan) read API.
- GitHub Secrets: **`AWS_ACCOUNT_ID`**, **`TF_VAR_DB_PASSWORD`** (cho job plan).

7.5 Rollback (SA)

Thành phần	Cách
Terraform	`git revert` + `apply` có kiểm soát
Worker	Push image tag đã biết ổn + refresh ASG; `deploy-aws` dùng `:latest` — cần kỷ luật tag/release cho prod nghiêm
Lambda	`update-function-code` với zip build từ commit cũ; `--publish` tạo version (alias là nâng cấp)

---

8. Hoạch định năng lực & "toán" cấu hình

8.1 Lambda ↔ RDS connection budget

- `db.t3.micro` thường ~**66** `max_connections` (AWS docs — kiểm tra theo thời điểm).
- Thiết kế giới hạn **đồng thời batch** SQS–Lambda: `lambda_event_source_max_concurrency` = **5** (không phải 5 Lambda tuyệt đối nếu account có throttle khác — nhưng là **governor** chính để không vượt DB).

Quy tắc tham chiếu:

```
effective_lambda_parallelism (chịu ESM cap)  ⟹  kiểm tra DatabaseConnections trên RDS
max_connections_plan ~ ceil(safety_factor × db_max_connections)  với safety_factor ~ 0.7–0.8
```

8.2 Visibility timeout

- Rule of thumb trong module queue: **VT ≥ 6 × lambda_timeout**.
- Hiện tại: **timeout Lambda 180s** → **VT 1080s** (`module.queue` + wire từ demo).

8.3 Throughput hàng đợi (lý thuyết)

```
records_per_sec ≈ (số invocations Lambda đang chạy × batch_size) / avg_duration_sec
với batch_size = 10, maximum_concurrency (ESM) = 5 (upper bound)
```

Giá trị thực tế phụ thuộc I/O DB, parse JSON, S3 Get.

8.4 Worker

- ASG **1–2** node; node thứ hai khi CPU kéo dài cao — cân bằng chi phí vs headroom.

---

9. Sẵn sàng cao, chịu lỗi & điểm đơn lỗi (SPOF)

9.1 Trạng thái Scope 1

Thành phần	HA / FT
Worker	ASG **Multi-AZ** (subnet private ≥ 2)
SQS	Dịch vụ managed, durable
Lambda	AZ-balanced trong vùng; giới hạn concurrency về mặt thiết kế
RDS	**Single-AZ** trong demo — **SPOF có chủ đích** (RTO/RPO phụ thuộc backup/snapshot — **không** cam kết HA DB)
NAT	**Đơn** — single AZ attachment; mất AZ có thể ảnh hưởng egress (nâng cấp: NAT/AZ)

9.2 Cơ chế chống lỗi ứng dụng

- Retry **SQS/Lambda** có kiểm soát; **DLQ** cô lập poison.
- **Idempotent** write khi at-least-once.
- **Claim Check** giảm lỗi do giới hạn kích thước message.

9.3 Lộ trình nâng cấp (SA backlog)

- RDS **Multi-AZ** (`db_multi_az`); **RDS Proxy** khi tăng concurrency app.
- Drill **backup/restore** định kỳ; alarm **FreeStorageSpace**.
- HTTPS (ACM) + redirect 80→443; WAF nếu public.

---

10. Khả năng quan sát — log, metric, cảnh báo, dashboard

10.1 Logging

Nguồn	Log group / export	Terraform

Lambda	`/aws/lambda/<project>-<env>-ingester`	`aws_cloudwatch_log_group.lambda`
Worker	`/ec2/<project>-<env>-worker`	`aws_cloudwatch_log_group.worker`
RDS	CloudWatch Logs **postgresql**, **upgrade**	`enabled_cloudwatch_logs_exports` trên `aws_db_instance.main`

Khuyến nghị SA: correlation id từ worker → message → Lambda (chuẩn hóa field log).

10.2 Metrics then chốt

- **SQS:** `ApproximateNumberOfMessagesVisible`, tuổi message cũ nhất (widget/main trong dashboard).
- **Lambda:** Invocations, Errors, Throttles, Duration.
- **RDS:** CPUUtilization, DatabaseConnections (dashboard + alarm CPU).
- **ASG/EC2:** CPU (scale), GroupInServiceInstances (alarm max).

10.3 Alarms (Terraform)

Alarm	Điều kiện	Resource
DLQ có message	SQS visible > 0 trên queue DLQ	`aws_cloudwatch_metric_alarm.dlq_has_messages`
Lambda errors	Sum Errors > 5 / 5 phút	`aws_cloudwatch_metric_alarm.lambda_errors`
RDS CPU	> 80% 5 phút	`aws_cloudwatch_metric_alarm.rds_cpu`
ASG tại max	InService ≥ max	`aws_cloudwatch_metric_alarm.asg_at_max`

10.4 Dashboard

- `aws_cloudwatch_dashboard.main` — tên `${project}-${environment}-overview` (ví dụ `crawler-demo-overview`): SQS main+DLQ, Lambda, RDS, ASG.
- **SNS:** `aws_sns_topic.alerts` + email subscription (**cần confirm** email sau `apply`).

10.5 Health endpoints

Kiểu	Cấu hình	Terraform
ALB → web	Path `/health`, matcher **200**, 30s	`aws_lb_target_group.web.health_check` (`environments/demo/main.tf`)
ASG	**EC2** status check	`aws_autoscaling_group.worker.health_check_type`

---

11. Checklist vận hành production

11.1 Trước deploy

- [ ] Điền `terraform.tfvars` (account, region, `db_password`, `alert_email`).
- [ ] Bucket S3 **state** tồn tại (bootstrap một lần — xem `README.md`).
- [ ] `terraform validate` / `plan` sạch; PR đã chạy `terraform-plan.yml` nếu đổi TF.
- [ ] File `infrastructure/aws/postgres_pure_layer.zip` tồn tại (Lambda layer).

- [ ] Tên `crawler-demo-*` trong GitHub Actions env khớp Terraform (hoặc đã đồng bộ).

11.2 Trong deploy

- [ ] `terraform apply` thành công; không destroy không mong muốn.
- [ ] Xác nhận subscription SNS (email).
- [ ] Sau image mới: ASG **instance refresh** (CI làm tự động trên `main`).

11.3 Sau deploy

- [ ] `output web_dashboard_url` truy cập được; `/health` 200 qua ALB.
- [ ] SQS depth dao động; Lambda không lỗi tăng đột biến (alarm).
- [ ] RDS có insert; không duplicate sai do conflict (quan sát).
- [ ] **DLQ = 0** hoặc đã có runbook replay/redrive.
- [ ] Dashboard hiển thị metric hợp lý.

11.4 Vận hành định kỳ & an toàn

- [ ] Rà IAM tối thiểu theo quý.
- [ ] Theo dõi chi phí **NAT Gateway**, data egress, RDS storage.
- [ ] Game day nhỏ: mô phỏng lỗi Lambda, worker stop, xử lý message DLQ.
- [ ] Khi production thật: bật `deletion_protection` /snapshot strategy trên RDS (hiện demo có thể `skip_final_snapshot` — **đổi trước prod** trong `modules/storage`).

Phụ lục A — Bản đồ nhanh khái niệm → resource Terraform

Khái niệm SA	Resource Terraform (tham chiếu `terraform state list`)
VPC / subnet / IGW / NAT / route	`module.networking.aws_vpc.main`, `aws_subnet.*`, `aws_internet_gateway.main`, `aws_nat_gateway.main`, `aws_route_table.*`
S3 VPC endpoint	`module.networking.aws_vpc_endpoint.s3`
KMS	`module.security.aws_kms_key.main`
Secrets DB	`module.security.aws_secretsmanager_secret.db`
SG worker/lambda/rds	`module.security.aws_security_group.worker`, `lambda`, `rds`
IAM Lambda / Worker	`module.security.aws_iam_role.lambda`, `aws_iam_role.worker`, profiles policies
SQS + DLQ	`module.queue.aws_sqs_queue.main`, `aws_sqs_queue.dlq`, policies
RDS + monitoring role	`module.storage.aws_db_instance.main`, `aws_iam_role.rds_monitoring`
S3 buckets	`module.storage.aws_s3_bucket.raw`, `aws_s3_bucket.exports`
Lambda + ESM	`module.lambda.aws_lambda_function.ingester`, `aws_lambda_event_source_mapping.sqs`
ECR + ASG + scale	`module.worker.aws_ecr_repository.worker`, `aws_autoscaling_group.worker`, alarms CPU

ALB stack	`aws_lb.web`, `aws_lb_target_group.web`, `aws_lb_listener.web_http`, `aws_autoscaling_attachment.web_tg`
Observability	`module.observability.aws_sns_topic.alerts`, `aws_cloudwatch_metric_alarm.*`, `aws_cloudwatch_dashboard.main`

Phụ lục B — Sự cố thường gặp & chỗ tra

Triệu chứng	Tra
`terraform init` lỗi bucket	`backend.tf` — tạo bucket + versioning trước
Lambda không consume / không decrypt	IAM `lambda_custom`; KMS key policy
Lambda timeout / duplicate feel	So khớp `lambda_timeout_seconds` vs `visibility_timeout_seconds`
Too many DB connections	Giảm `lambda_event_source_max_concurrency`; tăng class DB / Proxy (kiến trúc sau)
Worker không lấy image	IAM ECR; user-data; ASG refresh
DLQ tăng	Nội dung message; fix code; redrive có kiểm soát
Chi phí NAT	Xác nhận traffic S3 đi qua `aws_vpc_endpoint.s3`

Document control: bản tích hợp SA + Terraform + CI/CD trong một file — cập nhật khi đổi `project` / `environment`, đổi tên resource, hoặc thêm workflow. Để học lệnh Terraform từng bước, xem thêm `SERVICES_AND_TERRAFORM.md`.